EnsembleDroid: A Malware Detection Approach for Android System based on Ensemble Learning

REU fellows: Sharon Guan[1]

Faculty Mentors: Dr. Wenjia Li[2]

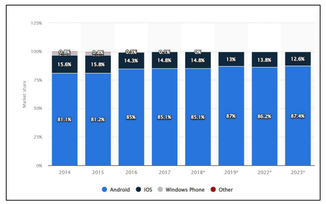
Affiliation: [1.] Stony Brook University, [2.] New York Institute of Technology

Emails: sharon.guan@stonybrook.edu, wli20@nyit.edu

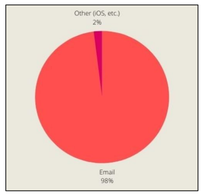
INTRODUCTION

In recent years, mobile devices such as smartphones and tablets have become one of the most popular digital devices of choice in our daily lives when it comes to functionality and convenience. With only ever increasing popularity, mobile devices with Android operating systems have become a common target for malware especially through third-party markets.

In this paper, we propose an ensemble learning based framework for detecting malware by using risky permissions as features to train a classifier and determine whether a mobile application (a.k.a. app) is malware or not. To evaluate the performance of the proposed malware detection approach, we have conducted a series of experiments using real world Android app datasets that are composed of both malicious and benign apps. Experimental results clearly show that the proposed malware detection approach can effectively detect malware with a high accuracy.



[1]



[2]

Fun Facts:

- 87% of the global market shares of smartphones are dominated by Android operating system
- As of March 2020, the total number of new Android malware samples amounted to 482,579 per month

Related Work

1. Static Analysis:
   - Analyzes issues without executing them.
   - Searches for the similarity between previously existing malware and the targeted application

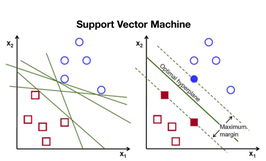
   Limitations: cannot recognize security susceptibilities during run time

2. Dynamic Analysis:
   - Analyzing for issues while an Android application is running in a safe environment.
   - Sandbox Environment or Smartphone devices

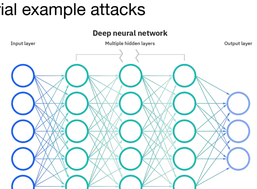
   Limitations: run time overhead

3. Machine Learning:
   - Application of AI that enables systems to learn and improve from experience without being explicitly programmed.
   - Does not require human supervision and can learn on its own

   Limitations: Prone to adversarial example attacks



[3]



[4]

METHODOLOGY

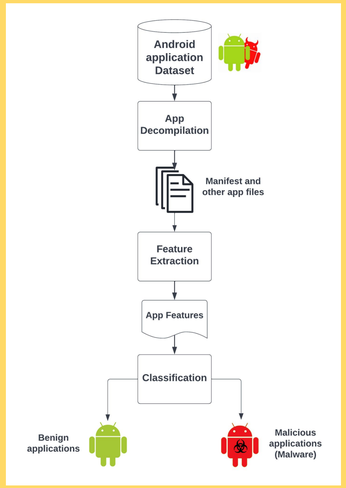


Figure 1. Overview of Detection Approach

Android Application Dataset:

- 5560 APKs from Drebin [5]
- 1550 APKs collected from Google Play Store

App Decompilation:

- Apktool [3] is used to decompiled apk files into other readable files, most notable the manifest (.xml) files

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.VIBRATE"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
```

Figure 2: Android Manifest Permissions

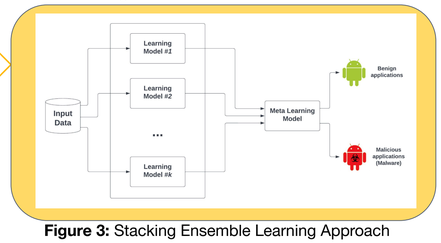


Figure 3: Stacking Ensemble Learning Approach

Feature Extraction:

- Android Permissions are extracted

App Features:

- Feature vectors are created using the permissions

Classification:

- Stacking Ensemble Learning Classification

RESULTS

```
# Defining homogeneous base model

level0 = list()
level0.append(('svm1',SVC(C=0.1, gamma = 0.1, kernel = 'rbf')))
level0.append(('svm2',SVC(C=0.1, gamma = 1, kernel = 'rbf')))
level0.append(('svm3',SVC(C=1, gamma = 0.1, kernel = 'rbf')))
level0.append(('svm4' ,SVC(C=1, gamma = 1, kernel = 'rbf')))
level0.append(('svm5', SVC(C=10, gamma = 0.1,kernel= 'rbf')))
level0.append(('svm6', SVC(C=10, gamma = 1,kernel= 'rbf')))
level0.append(('svm7',SVC(C=0.1, gamma = 0.1, kernel = 'linear')))
level0.append(('svm8',SVC(C=0.1, gamma = 1, kernel = 'linear')))
level0.append(('svm9',SVC(C=1, gamma = 0.1, kernel = 'linear')))
level0.append(('svm10', SVC(C=1, gamma = 1, kernel = 'linear')))
level0.append(('svm11',SVC(C=10, gamma = 0.1, kernel = 'linear')))
level0.append(('svm12',SVC(C=10, gamma = 1, kernel = 'linear')))

# Defining meta-model
level1 = MLPClassifier()

# Building stacking classifier
stack_clf = StackingClassifier(estimators = level0, final_estimator = level1, cv=10)
stack_clf.fit(X_train, y_train)
```

	precision	recall	f1-score	support
0	0.76	0.85	0.80	292
1	0.96	0.93	0.94	1052
accuracy			0.91	1344
macro avg	0.86	0.89	0.87	1344
weighted avg	0.91	0.91	0.91	1344

0: Number of Benign apps detected

1: Number of Malicious apps detected

Figure 4. Homogeneous Stacking Classifier and Results

```
# Building base model
level0 = list()
level0.append(('knn', KNeighborsClassifier(leaf_size= 10, metric ='manhattan', n_neighbors= 3, p= 1, weights= 'distance')))
level0.append(('svm', SVC(C= 10, gamma= 0.1, kernel= 'rbf')))
level0.append(('lr', LogisticRegression(C= 0.0001, max_iter= 100, penalty= 'none', solver= 'newton-cg')))

# Building meta-model
level1 = LogisticRegression()

# Stcking classifier
stack_clf2 = StackingClassifier(estimators = level0, final_estimator = level1, cv=10)
stack_clf2.fit(X_train, y_train)
```

	precision	recall	f1-score	support
0	0.77	0.83	0.80	292
1	0.95	0.93	0.94	1052
accuracy			0.91	1344
macro avg	0.86	0.88	0.87	1344
weighted avg	0.91	0.91	0.91	1344

Figure 5. Heterogeneous Stacking Classifier and Results

CONCLUSION

In this paper, we proposed EnsembleDroid, a malware detection approach using ensemble learning. Specifically, we explored both homogeneous and heterogeneous stacking models when deploying the ensemble learning algorithm. When we build the homogeneous stacking classifier, we use multiple support vector machines as the base models and the multilayer perceptron classifier. The heterogeneous stacking classifier uses the K-nearest neighbor, support vector machine, and logistic regression algorithms as the base models and logistic regression model again as the meta-model. During the experimental study, both models interestingly produced a 91% accuracy in the classification report.

DISCUSSION

Throughout the making of this paper, we realized some possibilities as to why our results produced a 91% accuracy.

- Due to the limited time duration of the summer research program, the dataset used for the experiments may be considered small.
- The dataset also does not maintain an ideal 50/50 ratio between benign and malicious apps.

FUTURE WORKS

Possible routes to improve accuracy:

- Increase dataset
- Make ratio of benign and malicious apps 50/50
- Add weights to the dataset if the ratio is not event
- Explore and utilize more learners for classification

For further into the future:

- Test classifier against adversarial example attacks

REFERENCES

[1] Published by Statista Research Department and J. 27, "Smartphone OS Market Share Forecast 2014-2023," *Statista*, 27-Jul-2022. [Online]. Available: https://www.statista.com/statistics/272307/market-share-forecast-for-smartphone-operating-systems/#:~:text=Smartphones%20running%20the%20Android%20operating,percent%20share%20of%20the%20market. [Accessed: 02-Aug-2022].

[2]R. Nishad, "Ultimate List of Cyber Security Stats, no.8 will shock you (2021)," *360 Tech Explorer*, 28-Aug-2021. [Online]. Available: https://360techexplorer.com/cyber-security-stats/. [Accessed: 03-Aug-2022].

[3] R. Gandhi, "Support Vector Machine - introduction to machine learning algorithms," *Medium*, 05-Jul-2018. [Online]. Available: https://towardsdatascience.com/support-vector-machine-introduction-to-machine-learning-algorithms-934a444fca47. [Accessed: 02-Aug-2022].

[4] By: IBM Cloud Education, "What are neural networks?," *IBM*. [Online]. Available: https://www.ibm.com/cloud/learn/neural-networks. [Accessed: 02-Aug-2022].

[5] D. Arp, M. Spreitzenbarth, M. Hubner, H. Gascon, K. Rieck, and C. E. R. T. Siemens. "Drebin: Effective and explainable detection of android malware in your pocket." In Ndss, vol. 14, pp. 23-26. 2014

ACKNOWLEDGEMENT

This research was supported by the National Science Foundation (NSF) Research Experiences for Undergraduates (REU) program. We would like to thank all of the mentors and research fellows at the New York Institute of Technology who have provided their helpful insight and expertise that greatly assisted with our research.We want to extend our thanks to Ziqing Yang and Michael Trzaskoma for their constant help over the program.

This project is funded by National Science Foundation Grant No. CNS-1852316 and New York Institute of Technology.